Green Software Project: Optimizing Zulip's Energy Consumption

Rens Weerman
s1091202

Artur Wiadrowski
s1090597

Michael Schrijer
s1098599

June 14, 2026

1 Overview

We chose the open-source project Zulip [12, 14, 9] which is a team chat designed as an alternative to other applications like Slack and Microsoft Teams. It aims to reduce communication clutter and improve the organization of conversations through its threading model. Unlike other applications, Zulip organizes conversations into topics within channels which makes it easier to follow multiple discussions simultaneously and catch up on missed conversations. Zulip is accessible across multiple platforms with native applications for Linux, Android, Windows, iOS and macOS as well as a web application and through a terminal.

Anyone can choose to host Zulip on their own servers which prevents vendor lock-in and gives full control over data. Zulip also provides integrations with popular tools like GitHub and Jira which enables teams to receive notifications and automate workflows through webhooks which allow developers and project managers to receive updates in Zulip without switching between applications. Its backend is built in Python using the framework Django and its web application is built with TypeScript and JavaScript.

2 Typical use cases

There are several use cases of Zulip. As communication tool, it provides several benefits unique to itself. For example, software developers have the incentive to use Zulip, because it provides code syntax highlighting which enables anyone to join a call without the need for the presenter to go over the previously sent code again. Another such example is the Rust programming community, where it has been found that the organization of threads within Zulip allows for much faster resolution of GitHub issues [7]. Another use case is broadcasting files to many people at once. At universities, files may be distributed according to what courses students follow, and there could be further divisions into groups. Zulip also has its own bot API. It turns out that this can be useful for using Large Language Models. However, one may occasionally want to have it learn

only based on the information provided within a certain thread. While this can be troublesome with other platforms such as Slack, it can be easily managed in Zulip.

3 Lifecycle analysis

For our lifecycle analysis, we decided to use the kWh impact metric in order to provide insights on the energy costs of Zulip's process. In addition, in order to tie this kWh metric to a use of the Zulip project, we will use messages sent or received as the useful work metric of our lifecycle analysis. The purpose of the lifecycle analysis is to understand the environmental impact of Zulip, by considering the project as a single process.

Our functional unit will relate to the metric of useful work defined earlier. We will try to consider the energy impact of sending/receiving messages between a Zulip client service and a Zulip server.

Within the lifecycle, we set a system boundary to include only the processes and services directly needed to support the functionality of the Zulip system as a whole. All these services should be considered within the lifecycle analysis to get a picture of Zulip's energy consumption and environmental impact.

In our analysis, we make the following assumptions:

- Our server hosting Zulip and its end users are located within the same region, limiting transmission costs.
- We assume it takes an average of thirty seconds for an end-user to create and send a message on the platform.
- We measure users on similar systems, taking both the Zulip application energy cost and the background cost of the desktop specified in the 2023 Assessment of the energy footprint of digital actions and services. This may be updated based on the measurements we make for Zulip before making any changes.

Based on our assumptions and the processes we have taken to be involved within the use of our functional unit, we can make an estimate for an electricity consumption below.

Process	**Energy consumption**
PostgreSQL queries [1]	$0.059\text{J} \cdot 100 \cdot 365 = 2153.5\text{J}$, 0.0005981944kWh
HTTPS requests [2]	$0.0000027\text{Wh} \cdot 100 \cdot 365 = 0.09855\text{kWh}$
Backend server [3]	1.47 kWh
End-users [3]	$\frac{104.39\text{kWh/year} \cdot 30\text{s}}{31536000\text{s}} = 9.93 \times 10^{-5}\text{kWh} \cdot 2 = 0.001984601\text{kWh}$
Total	$0.0005981944+0.09855+1.47+0.001984601 = 1.571\text{kWh}$

4 Energy processing diagram

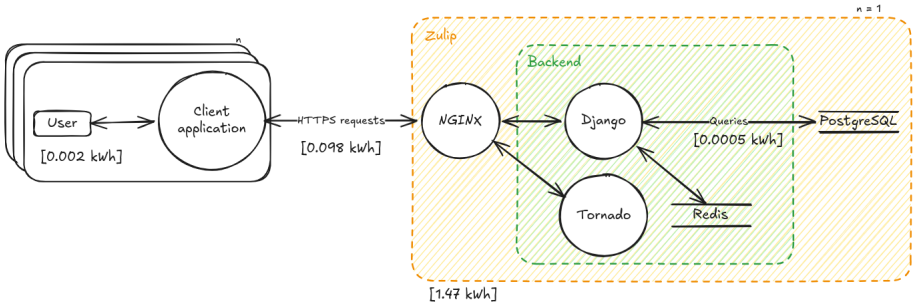

Figure 1: Energy processing diagram.

5 Relevant strategies

As shown in Figure 1, Django handles all logic, processing, making queries to the database and caching events. Therefore, we will focus on optimizing Django's energy consumption. According to Zulip's documentation [9], the endpoints for updating the presence status, fetching of messages and the homepage have the most average impact. Zulip defines the average impact as the multiplication of the request volume by average time which roughly corresponds to the load on the CPU the server runs on. Therefore, we decided to focus on these endpoints, since reducing their impact would also reduce the CPU usage which in turn should reduce the amount of energy consumed. We pick the strategies "transmit less", "less layers and (unused) features" and "update" or "use lean systems"/"fitting software/hardware", because we think the request volume that is indicated for the endpoints could be reduced and we suspect that there might be features or legacy code present that is not used any more. We also suspect that by updating the Python version that is used in the virtual environment, we can reduce the energy consumption since it uses Python 3.12 by default however newer Python versions have gotten significant performance improvements [4].

6 Relevant areas for improvement

Below we provide an overview of the areas we selected to improve as mentioned in Section 5

6.1 Python version

6.1.1 Area

The first area of improvement we will be focusing on is the usage of Python for Zulip's backend. Although other systems, such as the database system within

the backend, do exist, our idea is that we can generally improve the energy consumption of Zulip's backend by making adaptations to the software used in the first place.

Although quite general, the idea for this first area is that Python versions can influence the speed and cost of running Zulip. Different Python interpreters might have different features or functionalities, or might have made changes that can speed of processes. In the case of Zulip, having a Python interpreter with less functionality, or workings more specific to Zulip, could have a positive impact on the system's overall energy consumption.

6.1.2 Hypothesis

The idea for this area of improvement relates to the work shown in Grinberg's earlier work [4]. From this, we expect that we can replicate Grinberg's work within Zulip. This means we expect to see a difference in energy consumption within Zulip depending on which Python version is used to run the backend. We also expect to find a version with some amount of benefit over other versions, when comparing the results and that we can replicate the results from the paper by Rolf-Helge Pfeiffer [5] for Zulip.

6.1.3 Implementation of strategies

For this area of improvement, we aim to use the strategies "use lean systems", and "fitting software/hardware". By changing the Python version, we would like to find a version that best suits the requirements presented by Zulip. Similarly as the process taken in the article, we wanted to test if there would be any measurable change in the overall power consumption of Zulip, depending on the Python version being used to run the Zulip backend.

In order to check whether our hypothesis is correct, we make use of Zulip's CI/CD or GitHub Action, by making adaptations in order to run energy benchmarks for different Python versions. At the start of every workflow, we install Python and run the Zulip API tests for each Python version and measure the energy consumption using CodeCarbon 3.2.7 [6].

Our work metric is difficult to define, because we used the API tests provided by Zulip and are interested in the energy consumption of the entire system. The CodeCarbon benchmarks do allow us to estimate the energy consumption with the calculated corresponding emissions.

6.1.4 Measurements

Test name	Python 3.10 (kg CO_2)	Python 3.11 (kg CO_2)	Python 3.12 (kg CO_2)
`test_the_api`	4.79E-04	3.19E-04	3.18E-04
`test_generated_curl_examples_for_success`	1.22E-04	7.39E-05	7.22E-05
`test_js_bindings`	2.98E-05	1.73E-05	1.71E-05
`test_invalid_api_key`	7.96E-06	5.23E-06	4.56E-06
`test_user_account_deactivated`	7.83E-06	4.50E-06	4.49E-06
`test_realm_deactivated`	7.83E-06	4.50E-06	4.49E-06

Table 1: Emissions for different Python versions.

From the CodeCarbon measurements, we can also see that the Github Action was run in Wyoming, United States on a machine with an Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80GHz using 16 GB of RAM running Linux-6.17.0-1013-azure-x86_64-with-glibc2.39 and tracking mode `machine`. As a result, the carbon emissions estimated by CodeCarbon are impacted by the available energy sources for that location, and could be different depending on where a Zulip server is hosted.

Test name	Python 3.10 (kWh)	Python 3.11 (kWh)	Python 3.12 (kWh)
`test_the_api`	5.22E-04	7.56E-04	7.51E-04
`test_generated_curl_examples_for_success`	1.33E-04	1.75E-04	1.71E-04
`test_js_bindings`	3.24E-05	4.10E-05	4.05E-05
`test_invalid_api_key`	8.66E-06	1.24E-05	1.08E-05
`test_user_account_deactivated`	8.52E-06	1.06E-05	1.06E-05
`test_realm_deactivated`	8.52E-06	1.06E-05	1.06E-05

Table 2: Energy Measurements for different Python versions.

6.1.5 Discussion of results

Table 2 shows that the energy consumption of the given tests is very similar between Python 3.11 and Python 3.12, with the latter being the most efficient. A significant deviation from this is Python 3.10 performing better than those two versions for the test `test_js_bindings`. It is unclear as to why Python 3.10 would perform better in this respect than Python 3.12. The other results make it preferable to use Python 3.12.

6.1.6 Reflection on the strategy/hypothesis/area selection

We have tested Python versions 3.10, 3.11, and 3.12. This has been done by using Github Actions. In order to have the measurements run automatically

upon a commit, we used a `.yaml` file in which we specified different Docker images to be used, each with a different Python version.

Obtaining measurements by themselves proved challenging, since Zulip's development environment is virtualized within a Docker container using Vagrant. Because of this, access to some specific hardware components is limited, for example, the Linux command `perf` does not work, since it does not have the permission access the RAPL counter. To circumvent this, we used the tool CodeCarbon [6]. CodeCarbon does give energy consumption measurements. To use it, we embedded CodeCarbon's tracking code in a Python script. We instantiate an EmmisionsTracker object, start it just before running a function to be measured, and then stop it immediately after.

Since the entire system is run within a Virtual Machine, there will always be some calculation errors. However, we expect these to be negligible. TODO: Expand with why these are negligible.

6.2 Presence status

6.2.1 Area

The presence status in Zulip provides information about the status of users. The purpose of the presence function is that through status indicators it allows users to quickly identify who is active, idle or offline. These status indicators are displayed adjacent to user names in the user list. The presence feature works by tracking whether the Zulip client is open and in focus in the last 140 seconds [10].

6.2.2 Hypothesis

As indicated in the documentation of Zulip [11], the response of the endpoint that provides presence data still sends a legacy format that will be removed in the future. We expect that by removing the legacy format that is still present we can achieve some reduction in the energy consumption, since the legacy format is always computed alongside the modern format which requires more CPU usage and data that has to be transmitted.

6.2.3 Implementation of strategies

The useful work metric for the presence status is the number of user status updates within a given time period. The measurement method that we will be using is adding tracking code from CodeCarbon to the tests that check whether the presence status works correctly, similar to Section 6.1.3. In order to account for variability we will conduct five measurements for each test and take the average energy consumption and carbon emission.

Based on our hypothesis, we applied the strategies "transmit less" and "less layers and (unused) features". We did this by modifying the function `get_presence_backend` in `zulip/zerver/views/presence.py` and `get_presen`

ce_dicts_for_rows in zulip/zerver/lib/presence.py. We also had to modify some tests as they would otherwise fail if the legacy format is not present.

6.2.4 Before and after measurements

Table 3 and 4 show the before and after measurements that were performed on the energy consumption of the tests that are used for the presence status and their estimated emissions.

Test name	**Energy consumption (kWh)**	**Emissions (kg CO_2)**
date_logic	0.000003	0.000001
do_change_user_setting_presence_enabled	0.000003	0.000001
email_access	0.000003	0.000001
filter_presence_idle_user_ids	0.000003	0.000001
history_limit_days_api	0.000003	0.000001
invalid_presence	0.000003	0.000001
last_update_id_api	0.000004	0.000001
last_update_id_api_no_data_edge_cases	0.000003	0.000001
last_update_id_logic	0.000003	0.000001
new_user_input	0.000004	0.000001
ping_only	0.000003	0.000001
presence_disabled	0.000008	0.000003
query_counts	0.000003	0.000001
same_realm	0.000003	0.000001
single_user_get	0.000005	0.000002
user_presence_row_creation_simulated_race	0.000003	0.000001

Table 3: Before measurements.

Test name	Energy consumption (kWh)	Emissions (kg CO_2)
date_logic	0.000003	0.000001
do_change_user_setting_presence_enabled	0.000003	0.000001
email_access	0.000003	0.000001
filter_presence_idle_user_ids	0.000003	0.000001
history_limit_days_api	0.000003	0.000001
invalid_presence	0.000003	0.000001
last_update_id_api	0.000004	0.000001
last_update_id_api_no_data_edge_cases	0.000003	0.000001
last_update_id_logic	0.000003	0.000001
new_user_input	0.000004	0.000001
ping_only	0.000003	0.000001
presence_disabled	0.000008	0.000003
query_counts	0.000003	0.000001
same_realm	0.000003	0.000001
single_user_get	0.000005	0.000002
user_presence_row_creation_simulated_race	0.000003	0.000001

Table 4: After measurements.

Table 5 shows the differences between the before and after measurements.

Test name	Difference in energy consumption (%)
date_logic	0.0
do_change_user_setting_presence_enabled	0.2
email_access	0.1
filter_presence_idle_user_ids	0.1
history_limit_days_api	0.1
invalid_presence	0.0
last_update_id_api	0.1
last_update_id_api_no_data_edge_cases	-0.1
last_update_id_logic	-0.1
new_user_input	-0.8
ping_only	0.2
presence_disabled	0.2
query_counts	-0.1
same_realm	-0.2
single_user_get	0.9
user_presence_row_creation_simulated_race	0.0

Table 5: Differences between before and after measurements.

6.2.5 Discussion of results

As shown in Table 5, we were not able to achieve significant reductions across all tests, since most of the differences are zero or close to zero which is a negligible difference. We were able to achieve the most reduction in the test `single_user_get`. This test involves repeatedly fetching presence for a single user, therefore computing both the modern and the legacy format requires the most energy. Some differences are negative which could be caused by measurement noise and test variability, such as cache behaviour or garbage collection. This shows that the computational overhead of formatting the presence status in both the modern and legacy format at the same time is minimal in practice.

In our hypothesis we assumed that by removing the legacy format we would be able to reduce the energy consumption across all presence-related tests which is incorrect as we were only able to achieve a reduction in one of the tests. Even though the differences between before and after measurements are minimal, depending on the amount of calls to the endpoint that are similar to the `single_user_get` the improvement could still achieve significant savings. However, we did not test for this.

6.2.6 Reflection on the strategy/hypothesis/area selection

We encountered some challenges during the implementation of the strategies. For example, we have limited knowledge on Zulip's codebase, therefore we are not completely sure whether we missed code that still computes the legacy format or performs additional checks. We also noticed that CodeCarbon reported higher values for the energy consumption of tests that are executed using `./tools/test-backend` without the parameter `zerver.tests.test_presence`. We think this is caused by the fact that tests are executed in parallel, therefore we had to add the extra parameter to ensure that we are only measuring the energy consumption of tests that correspond to the presence status. From these challenges, we learned that is important to have a solid knowledge base on the codebase that has to be optimized, since it affects both the improvements and the measurements.

6.3 Message Fetching

6.3.1 Area

The final area of improvement we have selected for the Zulip is the process of fetching messages within the backend. In Zulip users can communicate through different channels and groups. Messages can be fetched by specifying the information about the channels, such as who is involved in the communication and the amount of messages to be collected, in a narrow [8]. These narrows allow the Zulip backend to construct the queries necessary for retrieving the messages an end-user expects.

With such chats, messages regarding recipients and senders is important, and any fetched message for such a channel will require additional metadata

information. We expect that this information is, however, redundant in more direct communication also offered by Zulip. Besides channels, users in Zulip can also communicate in private messaging, through direct messaging (DM). In these chat environments, information about senders and receivers is not necessary, since it is already clear to the end user which DM they are making a request for. As such, we can reduce the information returned by the server while retaining the same information, leading to a potential reduction in bandwidth usage between end-users and the Zulip backend.

6.3.2 Hypothesis

We expect that we can reduce the bandwidth usage between the client application and the Zulip backend by reducing the information sent when messages are fetched by a client. The work metric for this area of improvement would then involve a single message sent between two users. We aim to reduce the bandwidth usage for one message, allowing us to extrapolate the result to a larger fetch request.

With this hypothesis, any changes made would relate to two strategies "transmit less", and "batching". This would be done by ensuring message requests are kept together, or batched, in order to reduce the necessary information that would otherwise be sent with each message separately. This could also relate to reusing results, where we make use of the end-user already knowing which private message they are interested in to simplify message fetching in the backend.

6.3.3 Implementation of strategies

The main goal for this area of improvement is to reduce or remove any information in a request for messages that becomes unnecessary given the context of a direct message. To start, we took a look at the API for Zulip, which specifies the parameters for specifying a message GET request, and the return values that make up the response from the backend. Each response from the Zulip backend will include an array of messages, with following return values:

- `avatar_url`
- `client`
- `content`
- `content_type`
- `display_recipient`
- `edit_history`
- `id`
- `is_me_message1`

- `reactions`
- `recipient_id`
- `sender_email`
- `sender_full_name`
- `sender_id`
- `subject`

The initial values that we found were redundant, given the context of already knowing the direct message recipient and sender, were the `display_recipient` and sender information. The `display_recipient` holds information regarding the e-mail address and `full_name` of the user themselves, which is already known; and the sender will always be the same, so these have to be requested only once. In addition, the `avatar_url` specifies the URL for the image of the sender's avatar image, which can then also be excluded.

In Zulip itself, we made changes to the backend codebase in order to create an additional check on the narrow supplied for a message get request. If a narrow specifies a DM between two users, we can remove the `sender_email`, `sender_full_name`, `display_recipient`, and `avatar_url` from the message requests.

6.3.4 Before and after measurements

With these improvements, we can now take a look at the bandwidth usage of a single message before and after removing the redundant information.

Single Message fetch	**Object Length**	**Estimated energy consumption (kWh)**	**Estimated emissions (kg CO_2**
Before	703	1.575×10^{-7}	4.346×10^{-5}
After	285	6.384×10^{-8}	1.762×10^{-5}

Table 6: Before and after GET message reductions, for a single message, in JSON Object String length.

Our estimated energy consumption for the message fetch is calculated using estimates for the data transfer of bytes or kilobytes within an average network [13]. Based on this estimation and the length of the new message fetch, we were able to find an energy consumption and emission that reflects the reductions possible with the removal of redundant information in message objects.

6.3.5 Discussion of results

Overall, the measurements seen before and after our reductions for message fetching do support our hypothesis. We do need to consider that these reductions in bandwidth usage can be achieved specifically in the case of messages sent directly between two users. In other cases, we would not be able to make the exact same reductions, along the lines of already knowing who is involved in the communication. The results also do not completely consider the possible effect of message length. For the measured message seen above, a short message content was taken, with no submessages and no reactions to the content. As a result, our reductions could weigh heavier than would be seen in a text heavy message, or one with a lot of additional metadata.

For most messages, however, our reductions for private messaging would see a considerable reduction in included metadata. The reductions hold for multiple messages, meaning if a request was made for an entire history of messages between users. Therefore, the reduction in the message fetch size holds not just for a single message, as measured above, but holds for any amount of messages, which makes the reduction much more valuable. The scalability of this means that the hypothesis could hold for all direct messaging within Zulip, and could potentially be a considerable reduction in energy consumption.

6.3.6 Reflection on the strategy/hypothesis/area selection

In regards to the applied strategies, we think that aiming to transmit less does have the most relevance in regards to message fetching, and this is reflected in the results. Our hypothesis was also somewhat confirmed, although we are limited somewhat to the idea of direct messaging, which does mean we miss out on possible reductions within other messaging areas of Zulip. While our results did match with this hypothesis, we could have aimed for a broader goal in message fetching, which may have led to more concrete reductions in energy consumption. While we do see concrete bandwidth reductions, this specific area of improvement is very limited to the ideas set in our hypothesis.

Besides this, we also had difficulties measuring and testing our ideas to reduce the bandwidth of message fetching due to the size and scope of the Zulip. While we were able to cut specific information from message responses according to the information a user would already have, the Zulip backend would require a lot of updating in order to handle the reduced message formats. In practice, the removal of information in message responses leads to processes in Zulip breaking, and made measuring actual energy consumption difficult. This is also why we are only able to discuss an estimated bandwidth energy consumption, rather than using results of tests from Zulip itself.

7 Reflection

Ultimately, we did find that the process of applying the strategies for green software to Zulip was relatively effective. For each area of improvement using our energy processing diagram, we were able to come up with at least one, if not multiple, relevant strategies. An example would be our efforts to find a more suitable Python version, which was an area of improvement we decided on through the strategy of finding fitting software or hardware for a system. Without these strategies, we would have been more limited in our attempts to improve the energy consumption of Zulip.

We did find some difficulties while trying to apply the strategies. The main difficulty we had throughout the project was the difficulty of trying to measure Zulip. As discussed in Section 6.1.3, we ultimately settled on using CodeCarbon for our energy and emissions measurements, but this did result in some issues in trying to use different tools within a virtualized environment. Our project and efforts to apply the strategies for green software were somewhat slowed down by these difficulties.

Besides the issue of virtualization, we also had some problems with the size of Zulip. In the time frame we had for the project, it was difficult to get a complete or sufficient overview of Zulip. Without such a complete understanding or overview, we were sometimes limited in the changes we were able to make. For example, changing message fetching in the system would lead to issues arising throughout the system infrastructure, which was at times unmanageable because of the time frame.

With these insights, we would be more ready to tackle a similar project in the future. The main considerations we would make for a second project, or attempt, would have to do with the decisions we would make in regards to the actual project we would aim to make more sustainable. Zulip turned out to be somewhat too large and, in the case of its use of virtualization, a challenge to measure within the time frame of the project. As such, future projects should focus on selecting a project with a more overseeable structure. Our idea when we selected Zulip was to have a project with a large amount of potential areas for improvement, but in practice this turned out to be too difficult to manage and measure.

References

[1] Comparing the energy consumption of different database management systems. URL: https://luiscruz.github.io/course_sustainableSE/2025/p1_measuring_software/g8_database.html, Last visited on 14 June 2026.

[2] Hina Anwar, Dietmar Pfahl, and Satish Narayana Srirama. An investigation into the energy consumption of http post request methods for android app development. In *ICSOFT*, pages 275–282, 2018.

[3] European Commission, Directorate-General for Energy, Ramboll, Resilio, A. Louguet, M. Caspani, D. Pytel, A. Pirlot, M. Faura Rosendo, and H. Blanadet. *Assessment of the energy footprint of digital actions and services*. Publications Office of the European Union, 2023.

[4] Miguel Grinberg. Is Python really that slow? URL: https://blog.miguelgrinberg.com/post/is-python-really-that-slow, Last visited on 27 May 2026.

[5] Rolf-Helge Pfeiffer. On the Energy Consumption of CPython. In *Quality of Information and Communications Technology*, pages 194–209, Cham, 2024. Springer Nature Switzerland.

[6] Codecarbon Team. Track reduce Co2 emissions from your computing. URL: https://codecarbon.io/, Last visited on 27 May 2026.

[7] The Zulip team. Case study: Rust programming language community. URL: https://zulip.com/case-studies/rust/, Last visited on 27 May 2026.

[8] The Zulip team. Construct a narrow. URL: https://zulip.com/api/construct-narrow.

[9] The Zulip team. Install a Zulip server. URL: https://zulip.readthedocs.io/en/stable/production/install.html, Last visited on 27 May 2026.

[10] The Zulip team. Status and availability. URL: https://zulip.com/help/status-and-availability, Last visited on 27 May 2026.

[11] The Zulip team. Update your presence. URL: https://zulip.com/api/update-presenceupdate-your-presence, Last visited on 27 May 2026.

[12] The Zulip team. Zulip - organized team chat. URL: https://zulip.com/, Last visited on 27 May 2026.

[13] Sophia Willows. The cost of bandwidth: What's in a byte? URL: https://sophiabits.com/blog/the-cost-of-bandwidth, Last visited on 14 June 2026.

[14] The Zulip team. Zulip. URL: https://github.com/zulip (Source code repository), Last visited on 27 May 2026.